

AI-Assisted Heart Disease Prediction Project

Course: Artificial Intelligence | Final Examination

Scenario: HealthPlus Hospital needs an AI system to predict heart disease.

Live Streamlit Website: <https://heartdiseaseaimllabfinal.streamlit.app/>

Dataset UCI Cleveland	Problem Type Binary classification	Best Model Random Forest	Best Accuracy 93.44%
---------------------------------	--	------------------------------------	--------------------------------

1. Dataset Understanding

Target variable: target. It is made from target_raw. Value 0 means no heart disease and values 1-4 mean heart disease. This is a binary classification problem.

Numerical features: age, resting blood pressure, cholesterol, maximum heart rate, and oldpeak. Categorical features: sex, chest pain type, fasting blood sugar, resting ECG, exercise angina, slope, ca, and thal.

Important expected features are chest pain type, exercise angina, oldpeak, maximum heart rate, ca, thal, age, and slope. Two challenges are missing values in ca/thal and the small dataset size.

2. Exploratory Data Analysis

Item	Result
Raw rows and columns	303 rows, 14 columns
Clean rows and columns	303 rows, 15 columns
Missing values in raw data	6
Duplicate rows	0
Class distribution	{0: 164, 1: 139}

Class distribution means how many records belong to each target class. In this project, it shows how many patients have no heart disease and how many patients have heart disease. It is important because a highly unbalanced dataset can make a model look accurate while still missing the smaller class.

EDA plots show that the target is fairly balanced, older age and lower maximum heart rate are connected with disease, and chest pain type plus exercise angina are strong warning signs. The correlation matrix also shows useful relationships with oldpeak and thalach.



Figure 1: Class distribution. This chart compares patients with no heart disease and patients with heart disease. It confirms that both target classes have enough records for training and testing. Because the classes are fairly balanced, accuracy is more meaningful than it would be in a highly imbalanced dataset, but recall and F1-score are still important for a medical prediction problem.

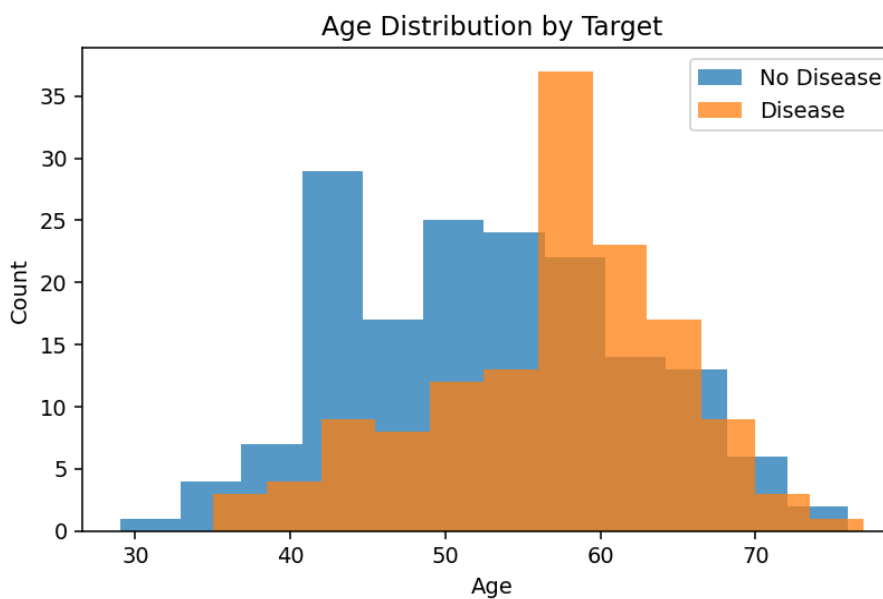


Figure 2: Age distribution by target. This chart shows how patient ages are spread across the disease and no disease groups. The disease group appears more common in older age ranges, while younger patients are more often in the no disease group. This supports keeping age as a model input and also explains why the engineered age_group feature can help the model learn age-related risk patterns.

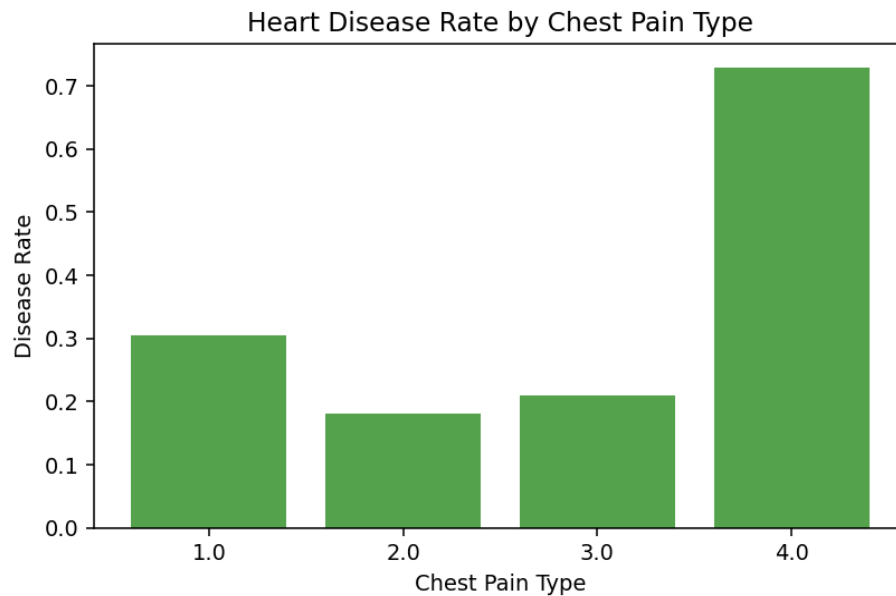


Figure 3: Heart disease rate by chest pain type. This chart calculates the disease rate inside each chest pain category instead of only counting records. Some categories have a much higher positive rate, which means chest pain type gives useful diagnostic information. Since the values are categories, one-hot encoding is used before model training so the algorithms can use this feature properly.

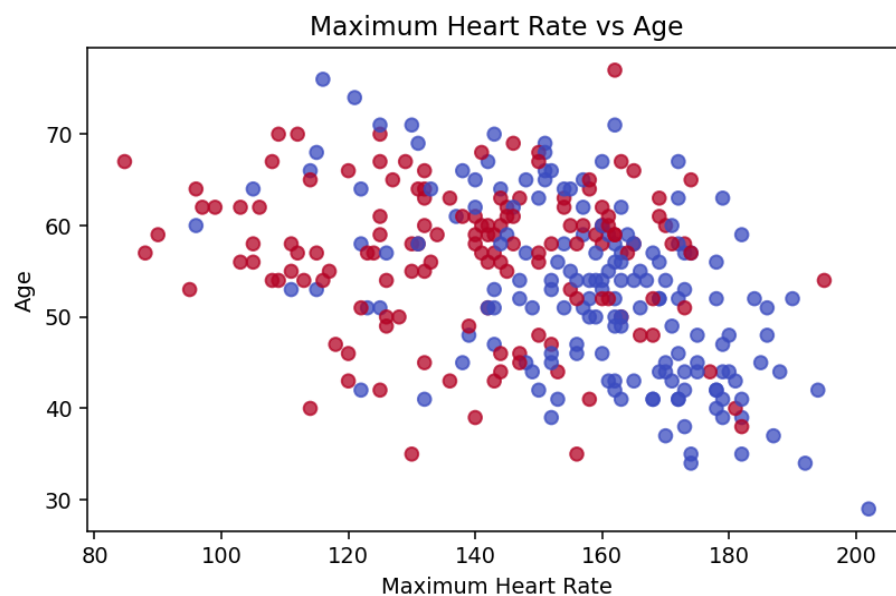


Figure 4: Maximum heart rate versus age. Each point represents one patient, with color showing the target class. The plot helps compare two important numeric features together. Many higher-risk patients are older and have lower maximum heart rate values, so the model receives both original features and the engineered heart_rate_gap feature to capture this relationship more clearly.

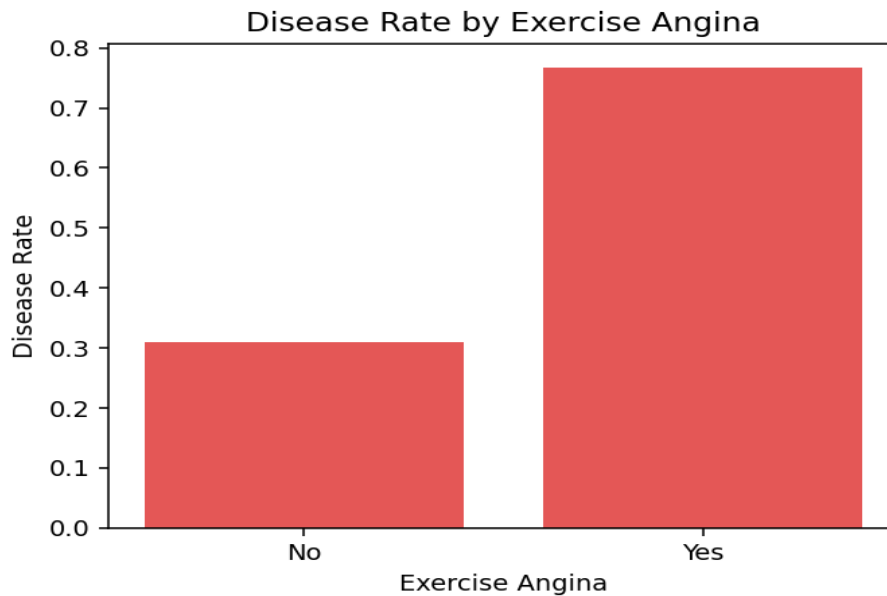


Figure 5: Disease rate by exercise induced angina. This chart compares patients who do and do not have exercise induced angina. The disease rate is higher for patients with exercise angina, which makes exang a strong warning feature. It is also included in the engineered risk_count feature because it adds useful clinical risk information.

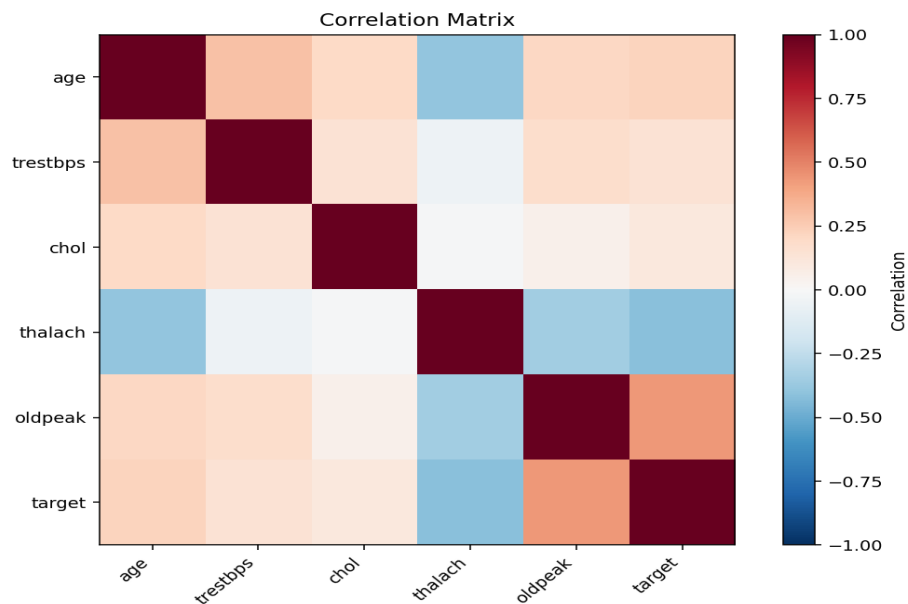


Figure 6: Correlation matrix. This heatmap shows the strength and direction of relationships between numeric features and the target. Positive colors mean two values tend to increase together, while negative colors mean one tends to decrease when the other increases. The matrix helps identify useful predictors such as oldpeak and thalach and also checks whether numeric inputs are too strongly related to each other.

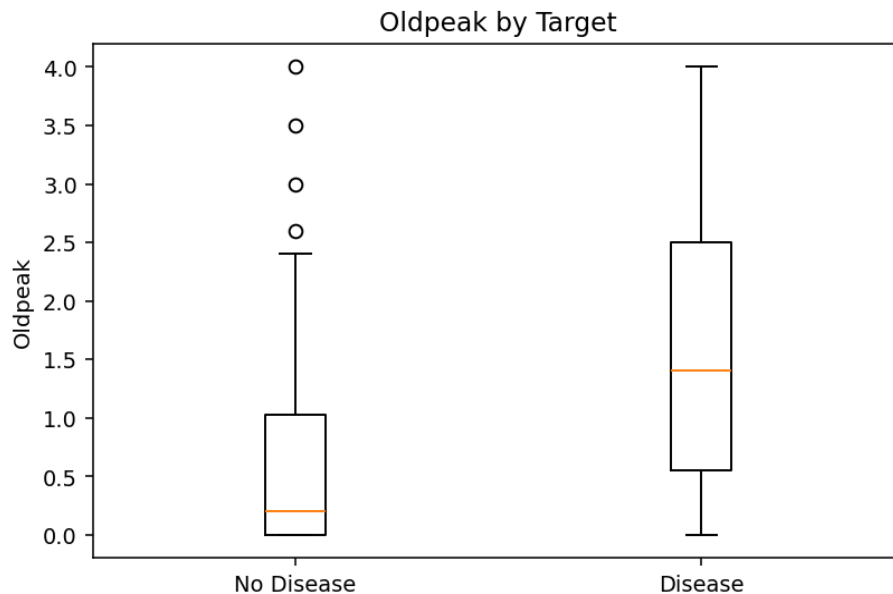


Figure 7: Oldpeak by target. This boxplot compares the oldpeak distribution for patients with and without heart disease. The middle line shows the median, the box shows the main spread, and outlying points show unusual values. Heart disease patients generally have higher oldpeak values, so oldpeak and oldpeak_high are useful predictors for the classification model.

3. Data Cleaning and Preprocessing

Missing numeric values were filled with the median because the median is less affected by extreme values. Missing categorical values were filled with the mode because it is the most common category.

Duplicates were removed. Numeric outliers were capped using the IQR method. IQR means Interquartile Range. Q1 means the first quartile, or the 25th percentile, so 25% of values are below Q1. Q3 means the third quartile, or the 75th percentile, so 75% of values are below Q3. IQR is calculated as $Q3 - Q1$. Values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ are treated as outliers and capped.

Categorical variables were one-hot encoded. Numeric variables were scaled because Logistic Regression and SVM work better when numeric values are on a similar scale.

4. Feature Engineering

Feature engineering means creating new useful columns from existing columns so the model can learn patterns more easily. It does not change the target answer; it only gives the model better input information.

New simple features were created: age_group, cholesterol_risk, bp_risk, heart_rate_gap, oldpeak_high, and risk_count. These features convert medical risk ideas into values that models can learn from.

Model	Baseline Accuracy	After Feature Engineering Accuracy
Logistic Regression	0.8689	0.8689
Random Forest	0.8689	0.9344
Support Vector Machine	0.8852	0.8689

5. Machine Learning Models

Logistic Regression: This model works like a simple scoring system. It gives weight to each feature, adds the scores, and then predicts the chance of heart disease. It is easy to understand and works well for many binary classification problems.

Random Forest: This model creates many small decision trees. Each tree makes a prediction, and the forest combines their votes. It often performs well because many trees reduce the risk of one weak tree making the final decision.

Support Vector Machine: SVM tries to find the best boundary that separates patients with heart disease from patients without heart disease. It is useful for medical classification because it can work well with smaller datasets.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.8689	0.8333	0.8929	0.8621	0.9675
Random Forest	0.9344	0.9	0.9643	0.931	0.9724
Support Vector Machine	0.8689	0.8333	0.8929	0.8621	0.961

Accuracy means the percentage of total correct predictions. Precision means, when the model predicts heart disease, how often it is correct. Recall means how many real heart disease patients the model successfully finds. F1-score is a balance between precision and recall. ROC-AUC shows how well the model separates disease and no disease classes.

Best model selected: Random Forest. It was selected by comparing recall, F1-score, ROC-AUC, and accuracy. SVM was included because it is commonly useful for medical classification when the dataset is not very large and a clear decision boundary is needed. In a hospital problem, recall is very important because missing a patient with heart disease is risky.

6. Model Evaluation

A confusion matrix is a table that shows correct and wrong predictions. For binary classification it has true negatives, false positives, false negatives, and true positives. In this medical task, false negatives are dangerous because they mean a patient has heart disease but the model predicts no disease.

Logistic Regression confusion matrix: $[[28, 5], [3, 25]]$. Accuracy=0.8689, Precision=0.8333, Recall=0.8929, F1=0.8621, ROC-AUC=0.9675.

Random Forest confusion matrix: $[[30, 3], [1, 27]]$. Accuracy=0.9344, Precision=0.9, Recall=0.9643, F1=0.931, ROC-AUC=0.9724.

Support Vector Machine confusion matrix: $[[28, 5], [3, 25]]$. Accuracy=0.8689, Precision=0.8333, Recall=0.8929, F1=0.8621, ROC-AUC=0.961.

Most appropriate metric: Recall, with F1-score also important. Recall reduces false negatives, while F1-score balances recall and precision.

7. Model Interpretation

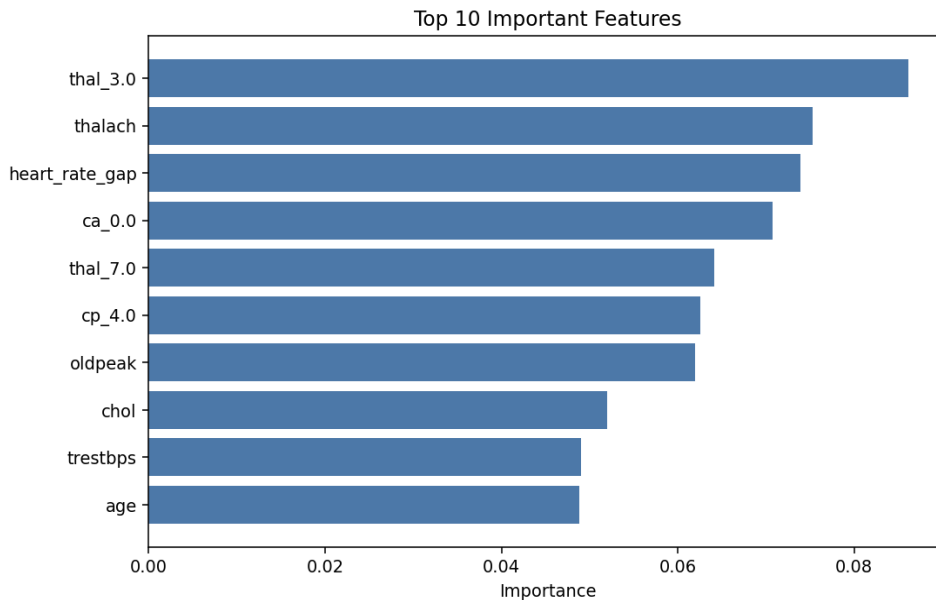


Figure 8: Feature importance. This chart ranks the top features used by the best model. Features with higher importance influenced the model prediction more strongly. The chart is useful because it connects the model result back to understandable medical inputs, helping explain why a prediction may be high or low instead of treating the model as a black box.

Feature	Importance
thal_3.0	0.0862
thalach	0.0753
heart_rate_gap	0.0739
ca_0.0	0.0707
thal_7.0	0.0641
cp_4.0	0.0626
oldpeak	0.062
chol	0.052
trestbps	0.049
age	0.0488

The top features are useful because they describe chest pain, exercise response, blood vessel information, heart rate, and ST depression. These are medically related to heart disease risk.

8. AI-Assisted Development Reflection

Prompt 1

Prompt used: Read the exam instruction PDF and list all required tasks and submission files.

AI output and my modification: AI identified the nine tasks and required files. I followed those sections in the script, notebook, report, and prompt history.

Prompt 2

Prompt used: Find a proper heart disease dataset for this project and explain the target variable.

AI output and my modification: AI suggested the UCI Cleveland Heart Disease dataset. I used the official UCI data and converted target_raw into binary target.

Prompt 3

Prompt used: Write easy Python code for cleaning, EDA, feature engineering, Logistic Regression, Random Forest, SVM, and evaluation.

AI output and my modification: AI suggested pandas, matplotlib, and scikit-learn. I used SVM because this is a medical classification task and kept pipelines for fair comparison.

Prompt 4

Prompt used: Create a short final report with section-by-section explanation, model results, prompts, limitations, ethics, and recommendations.

AI output and my modification: AI helped organize the report. I filled the report with actual results from the model run.

AI mistake noticed: AI first suggested some extra complex ideas, but I kept the code simple for an exam project.

My improvements: I used an official data source, added SVM for medical classification, checked the metrics from real model output, and added a separate AI prompt history file.

9. Critical Discussion and Recommendations

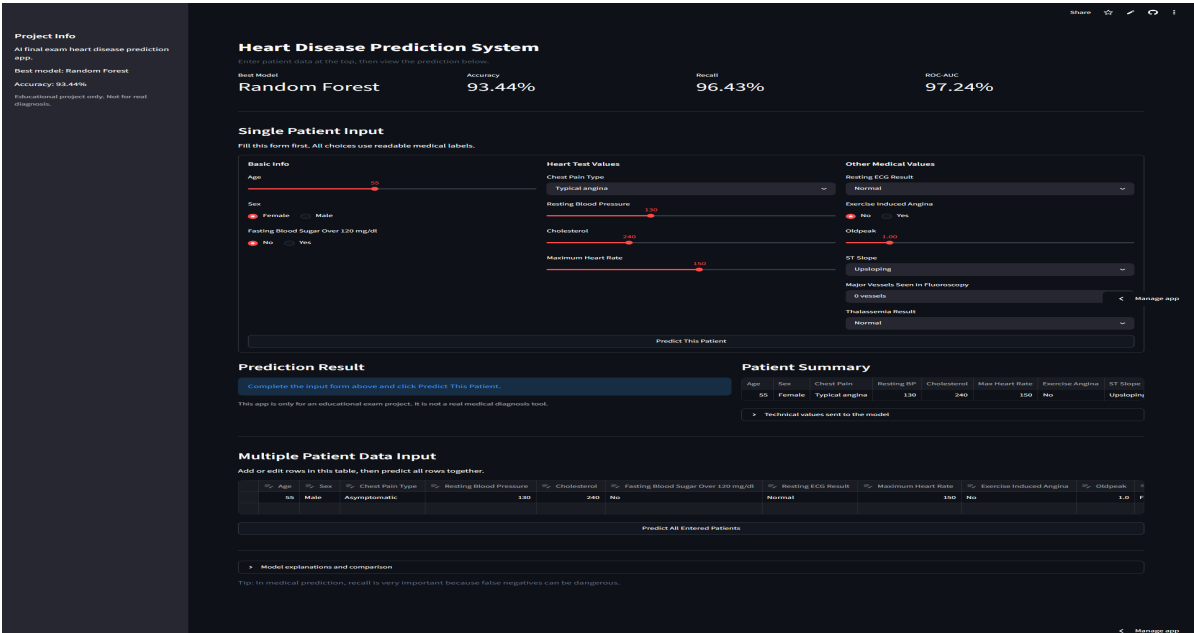
Limitations: the dataset is small and old, so results may not generalize to all hospital patients. Bias can happen if the patient group does not represent the real population. Data leakage was avoided by splitting train/test before model fitting inside pipelines.

Ethics and privacy: patient data must be protected, anonymized, and used only with permission. The model should support doctors, not replace doctors. For deployment, the hospital should validate the model on newer local patient data before real use.

10. Streamlit Web App and Deployment

The project is deployed as a working Streamlit app:
<https://heartdiseaseaimllabfinal.streamlit.app/>

The app turns the trained machine learning pipeline into an interactive prediction interface for single-patient and multi-patient testing.



Website screenshot: the deployed page shows a dashboard-style prediction system. The top area gives the selected model and evaluation scores, the center area collects patient information, and the lower area supports batch prediction for multiple rows.

Website Area	Purpose and Functionality
Model summary	Shows the best model, accuracy, recall, and ROC-AUC so the user can see the model quality before using predictions.
Single-patient form	Collects readable medical inputs using sliders, radio buttons, and dropdowns. The form is designed so users do not need to type raw model codes.
Prediction result	Displays whether heart disease risk is detected and shows the predicted probability when the model supports probability output.
Patient summary	Shows the selected values in a clean table and includes an expandable technical table with the numeric values sent to the model.
Batch input	Lets the user edit multiple patient rows, predict all rows together, view the results, and download a CSV output file.

Function Call Flow

How the app works internally: Streamlit runs `streamlit_app.py` as a Python web app. At startup, `load_model` reads `heart_disease_cleaned.csv`, builds the same preprocessing pipeline, trains the Random Forest model, and stores it in `st.cache_resource`. This cache keeps the app fast because the model is not rebuilt every time a widget changes. `load_metrics` reads `output/model_metrics.json` through `st.cache_data` so the dashboard metrics can be shown quickly.

Single-patient prediction flow: the user selects values in the form, clicks Predict This Patient, and Streamlit reruns the script. The readable labels are converted to numeric model values, `make_input_row` creates a one-row DataFrame, `add_features` creates `age_group`, `cholesterol_risk`, `bp_risk`, `heart_rate_gap`, `oldpeak_high`, and `risk_count`, then `model.predict` and `model.predict_proba` return the prediction and probability.

Batch prediction flow: the editable table accepts multiple patient rows. `readable_to_model_data` converts each readable row into the model's required column format. `predict_rows` adds the same engineered features for every row, calls the trained model, appends prediction labels and probabilities, and lets the user download the results as a CSV file.